

Entry points de Python

Integración con un pipeline de scikit-learn y respuestas sobre producción, gestores de paquetes y Makefile.

TALLER 2 | MACHINE LEARNING ENGINEERING

Proyecto: pipeline Iris con scikit-learn

Actividad preparada para contabilizar asistencia a la sesión del **02/10/2026**. El informe combina una investigación breve de la especificación de Python Packaging con su aplicación al código de este repositorio.

TRABAJO REALIZADO

Se revisó la declaración del comando **taller2-pipeline** en **pyproject.toml**, se agrego un Makefile para instalar el paquete y ejecutar ese comando, y se documento el uso con pip, uv, Poetry y Conda.

Hallazgo: el proyecto ya incluía la declaración del entrypoint de consola; la adaptación hizo explícito su uso desde la documentación y el flujo de Make.

EN ESTE INFORME

- Qué publica un entry point y cómo se ejecuta el pipeline instalado.
- Usos operativos en producción y compatibilidad con gestores de paquetes.
- Relación práctica entre el comando Python y los objetivos de Makefile.

1. Investigación y adaptación

La especificación de PyPA describe un entry point como metadatos con los que una distribución instalada anuncia componentes para que otras herramientas o aplicaciones sepan cómo encontrarlos. Sus tres partes son el grupo, el nombre y la referencia al objeto Python. [1]

Parte	Valor en Taller 2	Función
Grupo	console_scripts	Comando de consola
Nombre	taller2-pipeline	Nombre que se escribe en la terminal
Referencia	taller2_ml.pipeline:main	Función que se importa y se ejecuta

Cómo queda conectado al pipeline

El proyecto tiene un pipeline de clasificación Iris que escala variables con **StandardScaler** y entrena **LogisticRegression**. La función `main()` en `src/taller2_ml/pipeline.py` contiene el flujo de entrenamiento y evaluación. La tabla `[project.scripts]` de `pyproject.toml` publica esa función como comando instalable. [1][2]

```
[project.scripts]
taller2-pipeline = "taller2_ml.pipeline:main"
```

Al instalar el paquete, un instalador compatible con **console_scripts** crea un wrapper de consola. La invocación directa del módulo sigue disponible como alternativa. [1]

```
# Ambiente Python activado
make install
make run
```

```
# Equivalente directo
taller2-pipeline
```

```
# Alternativa de ejecución del módulo
python -m taller2_ml.pipeline
```

Se agrego **Makefile**: el objetivo **install** instala el paquete en modo editable en el Python activo y **run** llama al entrypoint. El README ahora explica el comando y sus formas de uso.

2. Respuestas: producción y paquetes

¿Qué usos tendría un entrypoint de Python en producción?

Un entrypoint de consola entrega una interfaz estable para iniciar una tarea Python desde un operador, un script de despliegue, un scheduler o un paso de CI/CD. En este proyecto, el comando instalado sería **taller2-pipeline**.

- Estandarizar el arranque del entrenamiento o de un job batch sin depender de una ruta local al archivo `.py`.
- Conectar el paquete a contenedores, tareas programadas y pipelines de despliegue usando el mismo comando.
- Exponer herramientas operativas como importables e instalar plugins por grupo de entry points; el consumidor puede descubrirlos con `importlib.metadata`. [1]
- Separar la interfaz de ejecución del código interno facilita cambiar la implementación sin cambiar el comando utilizado por operaciones.

Para producción, el entrypoint resuelve el punto de arranque. El equipo debe definir aparte configuración, registro de logs, manejo de errores, versiones de dependencias y cómo guardar artefactos del modelo.

¿Se puede usar con Conda, uv, Poetry, pip u otro gestor?

Si. El entry point describe metadatos del paquete; el gestor prepara el entorno y/o instala el paquete. Cuando la herramienta procesa esos metadatos y crea ejecutables, el comando queda disponible dentro del entorno instalado. [1][2]

Herramienta	Aplicación al proyecto
pip	Con <code>pip install -e .</code> , instala el proyecto y genera el comando de consola.
uv	<code>uv sync</code> sincroniza el proyecto; <code>uv run taller2-pipeline</code> lo ejecuta en su entorno. [3]
Poetry	Instala el proyecto con <code>poetry install</code> y ejecuta el comando mediante <code>poetry run taller2-pipeline</code> . [4]
Conda	En este repo, <code>environment.yml</code> instala el proyecto editable mediante pip dentro del entorno Conda. Para un paquete Conda nativo, <code>conda-build</code> admite <code>build.entry_points</code> . [5][6]

Con Conda, el archivo de ambiente organiza paquetes y aislamiento; el entrypoint llega como parte de la instalación del paquete Python (pip en esta configuración) o se declara en la receta si se construye una distribución Conda nativa. [5][6]

3. Relación entre entripoint y Makefile

Un Makefile agrupa comandos en objetivos, dependencias y recetas. DataCamp presenta Make como automatización local de tareas de proyecto y describe cómo trasladar pasos similares a GitHub Actions. [7] El entripoint y Make ocupan capas distintas del flujo: el primero publica un comando Python instalable; el segundo ofrece una forma corta de orquestar comandos.

Quién o qué	Acción	Resultado
Desarrollador o CI	make run	Make ejecuta la receta del objetivo run
Receta Make	taller2-pipeline	Invoca el wrapper instalado por el paquete
Wrapper de consola	taller2_ml.pipeline:main	Entrena y evalúa el pipeline Iris

Ejemplo de flujo para el repositorio

```
# Con el ambiente correcto activado
make install
make run
```

```
# Una tarea de CI puede ejecutar el mismo objetivo
make run
```

El ambiente debe estar activado antes de ejecutar Make, para que la receta use el Python que contiene las dependencias del proyecto. Tras instalar el paquete, también se puede ejecutar **taller2-pipeline** directamente.

Resumen

Los entry points hacen descubrible una función publicada por el paquete. pip, uv, Poetry o Conda instalan el paquete en un entorno adecuado; Make puede coordinar los pasos de instalación y ejecución. En Taller 2, el comando publicado es **taller2-pipeline** y Make lo ofrece como **make run**.

Fuentes consultadas

[1] PyPA, Entry points specification.

[2] PyPA, pyproject.toml specification.

[3] uv, Configuring projects: entry points.

[4] Poetry, Commands: console scripts.

[5] conda-build, Defining metadata: Python entry points.

[6] Conda, Managing environments.

[7] DataCamp, GitHub Actions and MakeFile: A Hands-on Introduction.

Consulta de fuentes: 04/10/2026.